



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en informática.

Asignatura: Lenguajes y Compiladores.

Tema 3

(Lenguajes y Gramáticas formales) .

Profesor:

Felix Marquez.

Estudiantes:

S1 Palma Franyibeth CI 31.564 .549.

S1 Rodríguez Edfrank CI 30.857.264

S2 Jesus Baez CI 26.753.871

S2 Javier Useche CI 28.700.959

Ciudad Guayana, Junio 2026.

INTRODUCCIÓN.

En la ciencia de la computación y la ingeniería de software, los lenguajes de programación y los sistemas de comunicación digital no son solo un conjunto de instrucciones. Son objetos matemáticos que requieren un manejo preciso y sistemático para estudiar, diseñar y ejecutar un lenguaje, es necesario dividirlo en sus elementos básicos y entender cómo se relacionan entre sí, un ordenador necesita herramientas teóricas para decodificar, validar y ejecutar estas instrucciones de manera eficiente estas herramientas son las gramáticas formales y los modelos de autómatas este informe de investigación examina cómo funcionan los lenguajes formales y los métodos algebraicos que se utilizan para optimizarlos en el primer bloque, se estudian los principios teóricos clásicos, basados en la Jerarquía de Chomsky se clasifican las clases de gramáticas y se vinculan a sus autómatas reconocedores correspondientes utilizando la notación Backus-Naur Form, en el segundo bloque, se aplica la abstracción matemática de las Gramáticas Libres de Contexto a un ámbito práctico se modelan secuencias geométricas fundamentadas en un alfabeto biológico cerrado para simular la creación de estructuras complejas mediante el código la viabilidad práctica de un lenguaje depende de su robustez operacional.

Por lo tanto, en el tercer bloque, se examinan las patologías gramaticales que ponen en riesgo la eficacia de los compiladores actuales, se presentan algoritmos matemáticos convencionales para solucionar estos problemas y se muestra cómo se puede convertir una Expresión Regular en un Autómata Finito Determinístico mediante un analizador léxico funcional en Python

A través de este recorrido conceptual y práctico, el equipo adquiere un entendimiento profundo sobre el papel que desempeña el formalismo matemático en la ingeniería de lenguajes.

BLOQUE 1. CONCEPTO FORMAL: ALFABETOS, LENGUAJES Y GRAMÁTICAS

Relación Gramática-Lenguaje

En ciencias de la computación y lingüística matemática, la relación entre una gramática y un lenguaje formal es de carácter generativo y descriptivo: la gramática es el conjunto finito de reglas (la especificación sintáctica) que define y produce la totalidad de las cadenas (habitualmente un conjunto infinito) que componen el lenguaje.

Para comprender esta relación formalmente, definimos:

1. **Alfabeto (Σ):** Un conjunto finito y no vacío de símbolos (por ejemplo, $\Sigma = \{a, b\}$).
2. **Palabra o Cadena (w):** Una secuencia finita de símbolos elegidos del alfabeto Σ . La cadena vacía se representa tradicionalmente con la letra griega ϵ (épsilon) o λ (lambda).
3. **Clausura de Kleene (Σ^*):** El conjunto de todas las palabras posibles de cualquier longitud (incluyendo la palabra vacía) que se pueden construir con los símbolos de Σ .
4. **Lenguaje Formal (L):** Un subconjunto de la Clausura de Kleene ($L \subseteq \Sigma^*$).

se define matemáticamente como una **Gramática Formal (G)** de la siguiente manera:

$$G = (V, \Sigma, P, S)$$

Donde cada uno de los elementos representa:

- **V (Conjunto de Variables o Símbolos No Terminales):** Un conjunto finito de elementos que actúan como variables intermedias y son sustituidos durante la derivación. Convencionalmente se escriben con letras mayúsculas.
- **Σ (Alfabeto o Símbolos Terminales):** Un conjunto finito de caracteres elementales que forman las cadenas del lenguaje final. Se cumple estrictamente la restricción de que ambos conjuntos son disjuntos: $V \cap \Sigma = \emptyset$.
- **P (Reglas de Producción):** Conjunto finito de reglas que definen las sustituciones permitidas. Tienen la forma general $\alpha \rightarrow \beta$, donde la cadena de la izquierda α pertenece a $(V \cup \Sigma)^* V (V \cup \Sigma)^*$ (es decir, contiene obligatoriamente al menos un no terminal) y la cadena de la derecha β pertenece a $(V \cup \Sigma)^*$.
- **S (Axioma o Símbolo Inicial):** Es el símbolo no terminal distinguido por el cual se inicia obligatoriamente todo proceso de generación ($S \in V$).

La relación entre ambos conceptos se establece a través del conjunto de palabras formadas exclusivamente por símbolos terminales que la gramática es capaz de generar. Este conjunto constituye el Lenguaje Generado por G, denotado como $L(G)$:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

Donde $\Rightarrow^*$ representa una secuencia de sustituciones (derivaciones) que parten de S y terminan en la cadena de terminales w .

Ejemplo Práctico de la Relación

Consideremos el lenguaje de expresiones lógicas booleanas simples compuestas por literales de verdad (t y f) y el operador de conjunción ($\&$).

- Alfabeto Terminal (Σ): $\Sigma = \{t, f, \&\}$
- Gramática (G):
- Variables (V): $\{E, Vval\}$
- Axioma (S): E

Reglas de Producción (P):

- $E \rightarrow Vval$
- $E \rightarrow E \& E$
- $Vval \rightarrow t$
- $Vval \rightarrow f$

Lenguaje generado $L(G)$: El conjunto de cadenas sintácticamente correctas que se derivan de las reglas:

$$L(G) = \{t, f, t \& t, t \& f, f \& t \& t, \dots\}$$

Cadenas mal formadas como $t \& o \& f$ no pueden ser generadas por ninguna combinación de las reglas de P , por lo que quedan excluidas del lenguaje.

Mecanismo de Generación (Derivación) Detallado.

Una gramática genera un lenguaje formal mediante un proceso iterativo de reescritura de cadenas conocido como derivación:

1. **Inicialización:** El proceso comienza obligatoriamente con el axioma inicial S .

2. **Sustitución Directa ($\Rightarrow$):** Si en una cadena actual αy existe una subcadena α que coincide con el lado izquierdo de una regla $\alpha \rightarrow \beta$ del conjunto P , se reemplaza α por β , obteniendo la nueva cadena βy . Esto se denota como: $\alpha y \Rightarrow \beta y$
3. **Sustitución en Múltiples Pasos ($\Rightarrow^*$):** Se aplican sucesivas reglas de producción de la forma: $S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n$. Cuando se realizan cero o más pasos de derivación para pasar de una cadena a otra, se denota con el operador transitivo $\Rightarrow^*$ (ej. $S \Rightarrow^* w_n$).
4. **Finalización:** El proceso de generación de una palabra concluye con éxito cuando la cadena obtenida contiene exclusivamente símbolos del alfabeto terminal ($w_n \in \Sigma^*$). Si aún quedan no terminales en la cadena, el proceso debe continuar hasta eliminarlos todos.

Jerarquía de Chomsky:

La Jerarquía de Chomsky clasifica las gramáticas formales en cuatro niveles o tipos (de 0 a 3) en función del grado de restricción aplicado a las reglas de producción $\alpha \rightarrow \beta$. A mayor tipo, las reglas son más restrictivas y el autómata necesario para su procesamiento es más sencillo.

Los 4 Tipos de Gramáticas:

1. **Tipo 0: Gramáticas Sin Restricciones (Recursivamente Enumerables):** Formato de Reglas: $\alpha \rightarrow \beta$, donde α debe contener al menos un no terminal ($\alpha \in (V \cup \Sigma)^* V (V \cup \Sigma)^*$) y β es una cadena cualquiera ($\beta \in (V \cup \Sigma)^*$). No existen limitaciones en la longitud de las cadenas a ambos lados de la regla.

Autómata asociado: Máquina de Turing.

- 2. Tipo 1: Gramáticas Sensibles al Contexto (Context-Sensitive):** Formato de Reglas: $\alpha A \beta \rightarrow \alpha \gamma \beta$, donde A es un símbolo no terminal, α y β representan el contexto circundante de variables o terminales, y γ es una cadena de símbolos no vacía ($\gamma \neq \epsilon$).

Restricción alternativa (No contracción): Para toda regla $\delta \rightarrow \eta$, la longitud del lado izquierdo es menor o igual a la del derecho ($|\delta| \leq |\eta|$), garantizando que la cadena no se reduzca de tamaño durante el proceso. Autómata asociado: Autómata Linealmente Acotado (LBA).

- 3. Tipo 2: Gramáticas Libres de Contexto (Context-Free):** Formato de Reglas: $A \rightarrow \beta$, donde el lado izquierdo contiene única y estrictamente un símbolo no terminal ($A \in V$) y el lado derecho es cualquier cadena ($\beta \in (V \cup \Sigma)^*$). La sustitución se realiza sin importar el contexto en el que se encuentre la variable. Autómata asociado: Autómata de Pila (PDA).

- 4. Tipo 3: Gramáticas Regulares:** Formato de Reglas: Reglas que siguen un orden lineal estricto. En el caso lineal derecho, las reglas permitidas son $A \rightarrow a$, $A \rightarrow aB$ y $A \rightarrow \epsilon$ (donde A, B son variables, a es un terminal y ϵ representa la cadena vacía).

Autómata asociado: Autómata Finito (Determinista o No Determinista - DFA o NFA).

Ejemplos Prácticos en Notación BNF (Backus-Naur Form):

La notación BNF utiliza los caracteres `< >` para encerrar a los símbolos no terminales y el operador `::=` para indicar la regla de producción. El carácter `|` funciona como un operador de opción (O).

Tipo 3: Gramática Regular (Identificadores del Sistema)

Esta gramática restringe el crecimiento de la cadena de forma lineal hacia la derecha, permitiendo procesar palabras simples.

BNF

```
<Identificador> ::= "x" <Sufijo> | "y" <Sufijo> | "z"  
<Sufijo>         ::= "1" | "2"
```

Explicación: Genera únicamente cadenas como `x1`, `y2` o `z`. Cumple la regla regular porque cada paso genera un terminal y deja, a lo sumo, un no terminal al final.

Tipo 2: Gramática Libre de Contexto (Estructura de Llaves Anidadas)

Permite modelar estructuras jerárquicas y ciclos de código donde unas instrucciones están dentro de otras.

BNF

```
<Bloque>         ::= "{" <Sentencias> "}"  
<Sentencias>     ::= <Instruccion> <Sentencias> | ""  
<Instruccion>    ::= "ejecutar_operacion;" | <Bloque>
```


Explicación: Permite la creación de bloques infinitamente anidados (por ejemplo, { ejecutar_operacion; { ejecutar_operacion; } }). Los autómatas regulares tipo 3 no pueden procesar estas estructuras porque carecen de memoria de pila.

Tipo 1: Gramática Sensible al Contexto (Validación de Tipo de Datos).

La regla se aplica considerando los elementos que rodean al símbolo, simulando un contexto de programación donde una variable requiere una declaración previa.

BNF

```
<Contexto_Entero> <Variable> ::= "int " <Variable>  
<Contexto_Texto> <Variable> ::= "string " <Variable>
```

Explicación: La sustitución o el reconocimiento de la palabra `<Variable>` cambia según el token o tipo de dato asignado a su izquierda, controlando las condiciones del entorno.

Tipo 0: Gramática Irrestricta (Modificación y Contracción Dinámica).

No posee limitaciones de tamaño o contexto, lo que permite transformar grupos enteros de símbolos en cadenas vacías o en estructuras totalmente diferentes.

BNF

```
<Compilador> <Error_Critico> ::= <Modo_Panico>  
<Modo_Panico> ::= ""
```

Explicación: Dos elementos complejos del Lado Izquierdo se reducen y contraen directamente a una cadena vacía en el Lado Derecho, modificando la estructura completa sin restricciones de longitud.

BLOQUE 2. DERIVACIÓN Y MODELADO

Fundamentos de los Lenguajes Formales en la Computación Gráfica:

El modelado estructural mediante texto se basa en asignar símbolos de un alfabeto a operaciones vectoriales simples. En ciencias de la computación, esto muestra que una secuencia de caracteres puede ser un objeto matemático que crea formas geométricas en un espacio bidimensional este enfoque proviene de los Gráficos de Tortuga, donde una máquina realiza movimientos en un lienzo leyendo caracteres de izquierda a derecha al ver el código como una lista de instrucciones de movimiento, los lenguajes formales se vuelven una herramienta matemática para diseñar. Esto permite crear estructuras espaciales usando teoría de la computación, asegurando que las figuras generadas tengan una sintaxis clara y predecible. De esta manera, el modelado estructural mediante texto ofrece una forma precisa de crear formas geométricas. Se basa en la relación entre símbolos y operaciones vectoriales los Gráficos de Tortuga son un ejemplo de cómo se puede usar este enfoque en ellos, una máquina sigue instrucciones escritas en forma de caracteres para crear figuras en un lienzo al usar este método, se puede garantizar que las figuras creadas sean claras y predecibles. Esto se logra mediante el uso de lenguajes formales y teoría de la computación, el modelado estructural mediante texto es una herramienta poderosa para crear formas geométricas de manera precisa y predecible. Se basa en la relación entre símbolos y operaciones vectoriales, y utiliza lenguajes formales y teoría de la computación para garantizar la claridad y precisión de las figuras generadas.

Limitaciones de los Lenguajes Regulares y la Necesidad de las Gramáticas Libres de Contexto (GLC)

En la jerarquía de Chomsky, las gramáticas se clasifican según lo que pueden expresar con sus reglas de producción. Los lenguajes regulares, que son del tipo 3, no son lo suficientemente poderosos para modelar cosas como geometrías complejas o fractales. Esto se debe a que no pueden manejar estructuras anidadas o balanceadas porque no tienen una memoria infinita para poder hacer cosas más complejas, como la recursividad profunda, se necesita una Gramática Libre de Contexto, que es del tipo 2. Esta gramática permite que una variable se expanda en varias ramificaciones independientes que pueden contener llamadas recursivas. Una Gramática Libre de Contexto se define como una cuádrupla matemática $G = (V, \Sigma, P, S)$

- V es el conjunto de variables o símbolos no terminales. Estos símbolos actúan como marcadores que definen la estructura de la figura, como la definición de un tronco, una rama o un peldaño Σ es el alfabeto de símbolos terminales, que son los caracteres reales que forman las cadenas. En este modelo, el alfabeto se limita a $\{a, c, g, t, [,]\}$.
- P es el conjunto de reglas de producción. Cada regla sigue la sintaxis $A \rightarrow \alpha$, donde la variable A se sustituye por una cadena α . La sustitución de A se hace de manera idéntica, sin importar los símbolos que la rodean en la cadena.
- S es el símbolo inicial o axioma del sistema, que es el elemento de V desde el cual comienza el proceso de transformaciones y derivaciones sintácticas. La Gramática Libre de Contexto es importante porque permite modelar estructuras complejas de manera efectiva.

El Mecanismo Computacional de Derivación por la Izquierda

La derivación es el proceso matemático mediante el cual, aplicando las reglas de producción, se transforma el axioma inicial S de forma progresiva hasta obtener una palabra compuesta únicamente por símbolos terminales de Sigma. Cuando una cadena intermedia produce directamente a otra, la transición se representa mediante " $\Rightarrow$ ". Si el proceso requiere de múltiples pasos jerárquicos, la relación se establece a través de la clausura transitiva " $\Rightarrow^*$ ".

En este modelo se utiliza estrictamente la derivación por la izquierda (leftmost derivation). En este procedimiento, en cada paso de sustitución, el sistema identifica la variable no terminal situada en el extremo izquierdo de la cadena y aplica la regla correspondiente. Este ordenamiento lógico es crítico en la informática gráfica, ya que garantiza que el árbol sintáctico se expanda en un orden secuencial que coincide con el recorrido de lectura que realizará el intérprete visual, permitiendo validar la consistencia de la figura antes de renderizarla en el lienzo.

Semántica Operacional del Alfabeto Genómico y Control de Pila

Para que la cadena final de ADN sea procesada por el motor gráfico, se dota a cada símbolo de una semántica operativa rígida que determina el comportamiento cinemático del agente de dibujo:

A) Movimientos Vectoriales Ortogonales (Desplazamiento Constante δ): Cada nucleótido representa un avance lineal equivalente a una unidad discreta de distancia δ en una dirección fija:

- El símbolo 'a' (Adenina) representa un desplazamiento hacia el Norte ($Y_{\text{nuevo}} = Y - \delta$).

- El símbolo 'c' (Citosina) representa un desplazamiento hacia el Este ($X_{\text{nuevo}} = X + \text{delta}$).
- El símbolo 'g' (Guanina) representa un desplazamiento hacia el Sur ($Y_{\text{nuevo}} = Y + \text{delta}$).
- El símbolo 't' (Timina) representa un desplazamiento hacia el Oeste ($X_{\text{nuevo}} = X - \text{delta}$).

B) Control Sintáctico de Bifurcaciones mediante Estructura LIFO (Pila): La rigidez de un alfabeto ortogonal impediría la creación de figuras que requieran ramificaciones complejas (como estructuras arbóreas), ya que el trazo se vería forzado a arrastrarse continuamente de regreso al origen. Para resolver esto, se integran dos metasímbolos basados en la lógica de los Sistemas de Lindenmayer, modelando el comportamiento de un Autómata con Pila:

- El carácter '[' ejecuta una operación Push. El intérprete detiene el flujo de dibujo y almacena las coordenadas actuales (X, Y) en el tope de una pila de tipo Last-In, First-Out.
- El carácter ']' ejecuta una operación Pop. El sistema extrae el último par de coordenadas de la pila y reubica instantáneamente el puntero de dibujo a dicha posición geográfica, realizando una transición limpia y sin generar trazo.

Especificaciones Matemáticas de las Gramáticas del Sistema

A) Gramática del Cuadrado (Geometría Cerrada Regular) Representa el modelo más simple de simetría ortogonal cerrada. No requiere el uso de variables secundarias ni operaciones de pila debido a que la trayectoria geométrica es continua, lineal y regresa a su origen.

- Especificación formal: $G = (\{S\}, \{a, c, g, t\}, P, S)$
- Regla de Producción: $S \rightarrow aaaaa ccccc ggggg tttt$

B) Gramática del Cubo (Superposición y Desplazamiento Lineal) Modela la perspectiva tridimensional mediante la generación de un cuadrado base, el almacenamiento del estado para realizar un desplazamiento diagonal escalonado y la duplicación simétrica de la estructura ortogonal cerrada.

- Especificación formal: $G = (\{S, C\}, \{a, c, g, t, [,]\}, P, S)$
- Reglas de Producción:
 1. $S \rightarrow C [a a c C] c c$
 2. $C \rightarrow aaaaa ccccc ggggg tttt$

C) Gramática del Árbol (Estructura Fractal Ramificada Binaria) Utiliza una expansión recursiva por la derecha y por la izquierda mediante corchetes balanceados para simular ramificaciones que se abren de forma simétrica desde un eje vertical común, terminando en nodos de follaje denso.

- Especificación formal: $G = (\{S, R, H\}, \{a, c, t, g, [,]\}, P, S)$
- Reglas de Producción:
 1. $S \rightarrow aaaaa [tttt aaaa R] ccccc aaaa R$
 2. $R \rightarrow t t a a H$
 3. $H \rightarrow [t][c][a]$

D) Gramática de la Escalera (Recursividad y Peldaños Equidistantes) Construye peldaños horizontales hacia la izquierda de forma iterativa y controla el paralelismo exacto entre los dos

rieles verticales de soporte, usando operaciones de pila para regresar al eje principal y bajando finalmente para completar el riel opuesto.

- Especificación formal: $G = (\{S, E\}, \{a, t, g, [,]\}, P, S)$
- Reglas de Producción:
 1. $S \rightarrow aaaa [ttt] E gggggggggggggg$
 2. $E \rightarrow aaaa [ttt] E$
 3. $E \rightarrow \text{vacío (épsilon)}$

E) Gramática de la Cruz (Simetría Radial Central) Demuestra cómo un punto central puede ser preservado como origen de coordenadas fijas mediante operaciones Push y Pop consecutivas e independientes, permitiendo la extensión simétrica de cuatro brazos ortogonales hacia los cuatro puntos cardinales del plano.

- Especificación formal: $G = (\{S\}, \{a, c, g, t, [,]\}, P, S)$
- Regla de Producción: $S \rightarrow [aaaaa][ggggg][ccccc][ttttt]$

BLOQUE 3: HIGIENE Y OPTIMIZACIÓN DE GRAMÁTICAS.

- **CASO A:** Ambigüedad en Gramáticas Formales y Multiplicidad Estructural

Definición Formal de Ambigüedad Sintáctica

Sea una gramática $G = (V, \Sigma, P, S)$. Se define una derivación por la izquierda (*leftmost derivation*) como aquella donde en cada paso de sustitución se reemplaza el no terminal situado más a la izquierda en la forma de frase. Una gramática G se clasifica matemáticamente como **ambigua** si existe al menos una cadena de terminales $\omega \in L(G)$ que admite dos o más árboles de derivación sintáctica estructurados de forma no isomorfa, o de manera equivalente, dos o más derivaciones por la izquierda diferenciadas.

La ambigüedad es una propiedad de la gramática, no del lenguaje. Existen lenguajes ambiguos que poseen gramáticas equivalentes no ambiguas.

Formalización de la Gramática Enferma (G_{amb})

Definimos la gramática recursiva simétrica para expresiones aritméticas que carece de restricciones jerárquicas:

$$V = \{E\}, \quad \Sigma = \{+, \times, id\}, \quad S = E$$

El conjunto de producciones P viene dado por:

$$E \rightarrow E + E \mid E \times E \mid id$$

Selección de la Cadena de Prueba e Inducción de Conflicto

Para demostrar la patología, se somete la gramática a la cadena de tokens:

$$\omega = id_1 + id_2 \times id_3$$

Demostración Algebraica por Doble Derivación por la Izquierda

A continuación se exponen las dos secuencias de derivación divergentes que demuestran la ambigüedad:

Derivación por la Izquierda 1 (D_1): Prioriza la estructura de la suma en la raíz del árbol, desplazando la multiplicación hacia las hojas.

- 1. $E \Rightarrow E + E$ (Por sustitución de $E \rightarrow E + E$)
- 2. $E \Rightarrow id_1 + E$ (Por sustitución del no terminal extremo izquierdo $E \rightarrow id$)
- 3. $E \Rightarrow id_1 + E \times E$ (Por expansión del segundo no terminal $E \rightarrow E \times E$)
- 4. $E \Rightarrow id_1 + id_2 \times E$ (Por sustitución del no terminal intermedio $E \rightarrow id$)
- 5. $E \Rightarrow id_1 + id_2 \times id_3$ (Por sustitución del no terminal final $E \rightarrow id$)

Derivación por la Izquierda 2 (D_2): Anida la suma dentro de la rama izquierda de la multiplicación, invirtiendo la precedencia algebraica estándar.

- 1. $E \Rightarrow E \times E$ (Por sustitución de $E \rightarrow E \times E$)
- 2. $E \Rightarrow E + E \times E$ (Por expansión del no terminal extremo izquierdo $E \rightarrow E + E$)
- 3. $E \Rightarrow id_1 + E \times E$ (Por sustitución del no terminal extremo izquierdo $E \rightarrow id$)
- 4. $E \Rightarrow id_1 + id_2 \times E$ (Por sustitución del no terminal intermedio $E \rightarrow id$)
- 5. $E \Rightarrow id_1 + id_2 \times id_3$ (Por sustitución del no terminal final $E \rightarrow id$)

Topologías de Árboles Sintácticos Abstractos (AST)

Como consecuencia directa, el sistema genera dos estructuras jerárquicas incompatibles:

Árbol Sintáctico S1 (Asociatividad Correcta)	Árbol Sintáctico S2 (Asociatividad Incorrecta)
[E]	[E]
/ \	/ \
[E] + [E]	[E] × [E]
/ \	/ \
id1 [E] × [E]	[E] + [E] id3
id2 id3	id1 id2

Impacto Crítico en la Arquitectura del Compilador

La ambigüedad causa que un programa semánticamente unívoco genere resultados binarios variables e impredecibles dependiendo del recorrido algorítmico que efectúe el *parser*, rompiendo la confiabilidad del sistema de ejecución.

- **CASO B: Eliminación de Recursividad por la Izquierda**

Fundamentación del Bucle Infinito en Parsers Top-Down

Si la gramática contiene una regla recursiva por la izquierda de la forma $A \rightarrow A\alpha$, la función asociada al no terminal A en un analizador descendente se invocará a sí misma de manera inmediata antes de consumir cualquier token. El autómata entra en una divergencia recursiva infinita (*stack overflow*).

Presentación de la Gramática Patológica

$$E \rightarrow E + T \mid T$$

Formalización del Algoritmo de Transformación

Para la regla recursiva izquierda inmediata: $A \rightarrow A\alpha \mid \beta$, se puede reescribir algebraicamente el sistema mutando la recursividad a la derecha mediante la introducción de una variable artificial extendida A' :

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

Aplicación Rigurosa Paso a Paso

- Símbolo crítico: $A = E$, cola recursiva $\alpha = + T$, caso base $\beta = T$.
- Regla principal transformada: $E \rightarrow T E'$
- Regla del símbolo auxiliar: $E' \rightarrow + T E' \mid \varepsilon$

Gramática Limpia Equivalente (G_{limpia})

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

$$T \rightarrow id$$

3.4. CASO C: Factorización por la Izquierda

3.4.1. El Problema del No-Determinismo

Si un no terminal posee múltiples alternativas que comparten un prefijo común, un analizador LL(1) no cuenta con suficiente información sintáctica para discriminar de forma unívoca qué rama tomar basándose únicamente en el token actual.

Presentación de la Gramática No-Factorizada

$S \rightarrow \text{if } C \text{ then } S \text{ else } S$

$S \rightarrow \text{if } C \text{ then } S$

$S \rightarrow \text{assignment}$

Prefijo común detectado: $\alpha = \text{if } C \text{ then } S$.

Proceso Matemático de Extracción de Factor Común

Aplicando la transformación $A \rightarrow \alpha A'$, derivamos los residuos hacia una variable complementaria S' , postergando la ramificación del *else*:

Gramática Resultante Optimizada

$S \rightarrow \text{if } C \text{ then } S \ S' \mid \text{assignment}$

$S' \rightarrow \text{else } S \mid \epsilon$

De esta forma, el análisis predictivo procesa el núcleo común determinísticamente y delega la opcionalidad del bloque *else* al estado sintáctico representado por S' .

Resolución Dinámica del Conflicto de Decisión

Bajo esta nueva configuración gramatical, el comportamiento del analizador sintáctico se torna determinista:

1. Al recibir en el flujo el token inicial if, el parser selecciona unívocamente la producción $S \rightarrow \text{if } C \text{ then } S \ S'$.
2. Consume la expresión de control C, la palabra clave then y la sub-sentencia sintáctica interna S.
3. Una vez completado este subárbol, el estado de evaluación se posiciona sobre la variable diferida S'.
4. En este punto, el lookahead de un token inspecciona el flujo de entrada:
 - Si el siguiente token es exactamente else, se expande la producción determinista $S' \rightarrow \text{else } S$.
 - Si el token corresponde a cualquier otro símbolo de la gramática (por ejemplo, el inicio de otra asignación o un delimitador de bloque), se asume el cierre de la condicional simple mediante la regla de anulación $S' \rightarrow \epsilon$.

La factorización por la izquierda preserva el determinismo sintáctico sin alterar las restricciones morfológicas o semánticas del lenguaje original.

BLOQUE 4: DE LA EXPRESIÓN AL AUTÓMATA.

En el desarrollo de compiladores, el análisis léxico es la primera fase. Esta fase convierte un flujo de caracteres en unidades que tienen significado sintáctico, llamadas tokens cuando se trabaja con lenguajes de propósito específico, como la Notación de Partidas Portable, las cadenas representan movimientos dentro de un tablero de ajedrez, según la Jerarquía de Chomsky, estas estructuras morfológicas corresponden a un Lenguaje Regular esto lo que significa que pueden ser descritas formalmente mediante Expresiones Regulares y reconocidas de forma óptima por Autómatas Finitos deterministas sin necesidad de una memoria de pila.

Definición del Subconjunto Simplificado del PGN:

Para garantizar la viabilidad computacional y la construcción de un autómata estrictamente finito y determinístico, se delimita un subconjunto del estándar PGN. Este subconjunto se enfoca en movimientos básicos de piezas y peones, excluyendo comentarios, enroques, marcas de fin de partida o promociones. Las reglas sintáctico-morfológicas del lenguaje regular propuesto se definen formalmente bajo las siguientes consideraciones:

- **Movimientos de Peón:** Se representan mediante la casilla de destino, combinando una columna en minúscula del alfabeto y una fila numérica del tablero. Por ejemplo, e4 o d5.
- **Movimientos de Piezas Mayores:** Se antepone la inicial de la pieza en mayúscula, seguida inmediatamente de la casilla de destino. Por ejemplo, Nf3 o Be6.

- Capturas: Se denotan intercalando el carácter de acción de captura antes de la casilla destino. En caso de capturas ejecutadas por peones, se antepone la columna de origen. Por ejemplo, Bxf7 o exd5.
- Declaración de Jaque: Cualquier movimiento que cumpla con los criterios anteriores y ponga en peligro inmediato al rey adverso puede finalizar de manera opcional con el meta símbolo +. Por ejemplo, Qh5+ o Nf3+.

La Expresión Regular se utiliza para definir los criterios textuales de manera formal en un patrón algebraico regular. Esto se hace mediante la sintaxis estándar de expresiones de programación, que se ve así:

`^[KQRBN]?[a-h]?x?[a-h][1-8]\+?$`

Para entender mejor esta expresión, la desglosamos en sus componentes:

- El símbolo ^ y el símbolo \$ son metacaracteres de anclaje que indican el inicio y el final de la cadena de entrada.
- [KQRBN]? es un cuantificador opcional que verifica si hay una o ninguna instancia de un identificador de pieza mayor.
- [a-h]? es un cuantificador opcional que representa la columna de procedencia del peón, necesario solo en escenarios de captura lineal.
- x? es un operador condicional que verifica si la jugada implica la captura física de una pieza del oponente.
- [a-h][1-8] es el núcleo obligatorio que conforma las coordenadas de la casilla de destino en el tablero.

- $\backslash + ?$ es un carácter de escape opcional que permite registrar la condición de jaque al final de la cadena de caracteres.

Para reconocer este lenguaje con memoria limitada, se define un Autómata Finito Determinístico (AFD) equivalente como una quintupla matemática $M = (Q, \Sigma, \delta, q_0, F)$, donde:

- Q es el conjunto finito de estados de control, que incluye $q_0, q_1, q_2, q_3, q_4, q_5, q_6$ y q_{err} .
- Σ es el alfabeto de entrada, que incluye letras y números que representan las piezas y las casillas del tablero.
- q_0 es el estado de arranque o inicial.
- F es el conjunto de estados finales o de aceptación, que incluye q_4 y q_6 .

Matriz de Transición de Estados (δ):

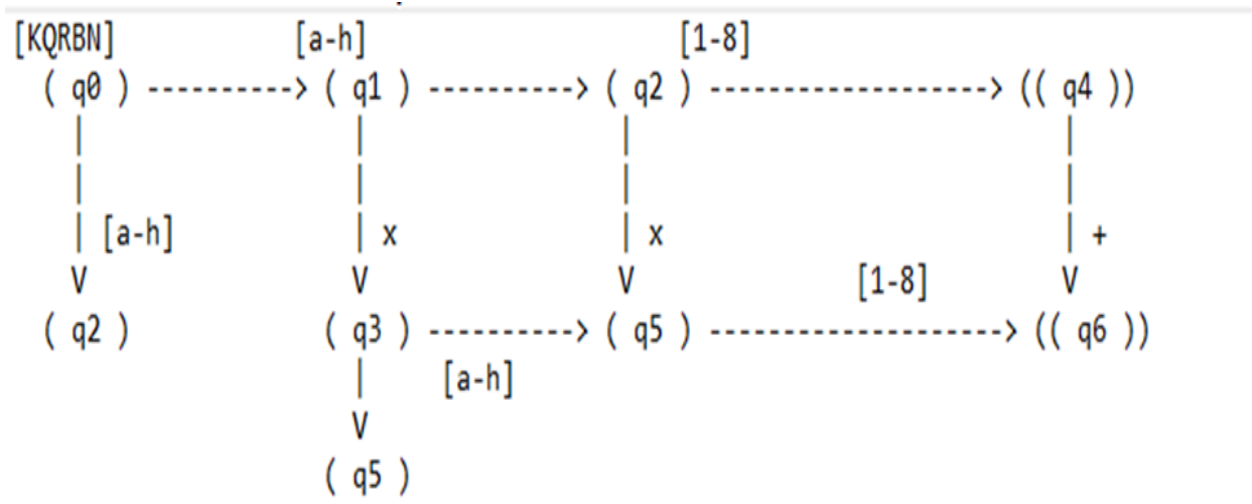
La función de transición de estados δ asigna un nuevo estado de manera determinista. Esto sucede cuando se pasa de un estado a otro según el carácter que se lee:

$\delta: Q \times \Sigma \rightarrow Q$

En otras palabras, δ indica a qué estado se va al leer un carácter. Esto se hace de forma secuencial, por ejemplo, si tenemos un estado q en Q y un carácter a en Σ , $\delta(q, a)$ determinará el siguiente estado así, la función δ nos permite saber en qué estado estamos después de leer un carácter.

Estado Actual	Entrada: [KQRBNI]	Entrada : [a-h]	Entrada : x	Entrada: [1-8]	Entrada: +	Cualquier Otro
q0 (Inicial)	q1	q2	qerr	qerr	qerr	qerr
q1	qerr	q2	q3	qerr	qerr	qerr
q2	qerr	q5	q3	q4 (Aceptación)	qerr	qerr
q3	qerr	q5	qerr	qerr	qerr	qerr
q5	qerr	qerr	qerr	q4 (Aceptación)	qerr	qerr
q4 (Aceptación)	qerr	qerr	qerr	qerr	q6 (Aceptación)	qerr
q6 (Aceptación)	qerr	qerr	qerr	qerr	qerr	qerr
qerr (Muerto)	qerr	qerr	qerr	qerr	qerr	qerr

Diagrama de Transición:



CONCLUSIÓN.

Este informe de investigación técnica muestra que las gramáticas formales y la teoría de autómatas son fundamentales para la ingeniería algorítmica y el pensamiento computacional, al estudiar la Jerarquía de Chomsky se puede entender la complejidad de los problemas sintácticos al conocer los límites de cada tipo de gramática, los ingenieros pueden elegir el modelo computacional más eficiente y económico para procesar datos, un hallazgo importante en este proyecto fue la importancia de la higiene y optimización gramatical. Los analizadores sintácticos dependen de estructuras lineales claras. Problemas como la ambigüedad pueden causar errores en el parser. La aplicación de algoritmos para eliminar y extraer factores comunes puede resolver limitaciones de diseño sin alterar el lenguaje original, la relación matemática entre las Expresiones Regulares, las Gramáticas Regulares y los Autómatas Finitos Deterministas demuestra que se puede utilizar software automatizado para validar lenguajes complejos. En resumen, para los estudiantes de Ingeniería en Informática, dominar estos fundamentos teóricos es clave para diseñar compiladores eficientes, intérpretes robustos y sistemas de análisis de datos optimizados. Esto ayuda a conectar la lógica matemática con la ingeniería de software de alto rendimiento.

REFERENCIAS BIBLIOGRÁFICAS

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3ra ed.). Pearson Educación.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Python Software Foundation. (2026). re — Regular expression operations. Python 3.13 Documentation. <https://docs.python.org/3/library/re.htm>
- Garshol, L. M. (2008). *BNF and EBNF: What are they and how do they work?* CoDeS. <http://www.garshol.priv.no/download/text/bnf.html>